SERVICENOW MICRO PROJECT REPORT

Implement Client Script & UI Policy (Incident)

1. Introduction

1.1 Project Overview

The Implement Client Script & UI Policy (Incident) project demonstrates how ServiceNow client-side controls can be used to enforce data integrity on Incident records. UI Policies and Client Scripts control field behavior, set values automatically, and validate information before a record is submitted.

1.2 Purpose

Incident records require consistent and accurate data entry for effective triage, routing, and resolution. Manual checks can lead to incomplete, inconsistent, or incorrect data, affecting reporting accuracy, SLA compliance, and service quality.

2. Ideation Phase

2.1 Problem Statement

Relying only on user awareness and manual checks can lead to incomplete, inconsistent, or incorrect Incident data being submitted, impacting reporting accuracy and SLA compliance. Standardized UI Policies and Client Scripts are required to control field visibility, mandatory status, and save-time validation.

2.2 Empathy Map Canvas & Brainstorming

Empathy Map: The source project document does not specify additional empathy-map details.

Brainstorming Ideas: Make Assignment group mandatory for High impact Incidents, make Urgency read-only when Impact is High, automatically set Urgency to High, prevent saving when Assigned To is empty for a High impact Incident, and prevent State changes through list editing.

3. Requirement Analysis

3.1 Solution Requirements

- ServiceNow instance with administrative or configuration access on the Incident table.
- UI Policy named “High Impact Control”, active, with condition *Impact is 1 – High*.
- UI Policy Action: Assignment group is mandatory (Reverse if false = true).
- UI Policy Action: Urgency is read-only when condition is met.
- onChange Client Script on `impact` to set Urgency to 1 when Impact is 1.
- onSubmit Client Script to prevent saving when Impact is 1 and Assigned To is empty.
- onCellEdit Client Script for `state` to block list editing.

3.2 Technology Stack

PLATFORM **ServiceNow**	CONFIGURATION **UI Policies & Actions**
SCRIPTING **onChange, onSubmit, onCellEdit**	TARGET TABLE **Incident (incident)**

4. Project Design & Planning

4.1 Proposed Solution & Architecture

The solution utilizes ServiceNow Incident forms and lists. UI Policies and UI Policy Actions dynamically govern field attributes, while Client Scripts handle automation, validation rules, and list-edit restrictions to ensure consistent data intake.

4.2 Project Scheduling

Configuration follows an iterative validation sequence: create UI Policy rules, deploy client-side scripts, verify mandatory field enforcement, test conditional reversibility, and confirm list-edit restrictions.

6. Functional Testing

Test Case	Action / Input	Expected Result
Test 1: Mandatory Enforcement	Set Impact = High, leave Assigned To empty, submit.	Record not saved; error message displayed for Assigned To.
Test 2: Successful Save	Populate Assigned To and submit.	Saves successfully; Urgency auto-sets and locks read-only.
Test 3: Reverse Condition	Change Impact from High to Medium.	Assigned To optional; Urgency becomes editable.
Test 4: List Edit Blocking	Modify State directly from Incident list view.	Alert triggers; State value remains unchanged.

11. Appendix: Source Code

onChange Script (Auto-set Urgency)

```
function onChange(control, oldValue, newValue, isLoading) { if (isLoading || newValue == '') { return; } if (newValue == '1') { g_form.setValue('urgency', '1'); g_form.addInfoMessage('Urgency set to High for High impact incident.'); } }
```

onSubmit Script & onCellEdit Script

```
function onSubmit() { if (g_form.getValue('impact') == '1' && g_form.getValue('assigned_to') == '') { g_form.showErrorBox('assigned_to', 'Assigned To is mandatory for High impact incidents.'); return false; } return true; } function onCellEdit(sysIDs, table, oldValues, newValue, callback) { alert('State cannot be updated using list editing. Please open the Incident.'); callback(false); }
```